GitHub Actions
Complete Study Guide

Based on Real-World Monorepo CI/CD Implementation

Smart CI · Docker · GHCR · Helm · Kubernetes · Real-World Issues & Fixes

■ Table of Contents

1. What is GitHub Actions?

GitHub Actions is a **CI/CD platform built into GitHub**. It lets you automate tasks triggered by events in your repository.

What you can automate	Example
Build & test on every push	Run mvn test on push to main
Build & push Docker images	docker build → ghcr.io
Update Helm/K8s manifests	sed image tag in values.yaml
Deploy to cloud	kubectl apply or helm upgrade

Everything is defined in **YAML files** stored in `.github/workflows/` inside your repo.

2. Core Concepts

Concept	Description
Workflow	The entire automation — defined in a .yml file
Trigger	What causes the workflow to run (push, PR, schedule, manual)
Job	A group of steps that run on the same machine (runner)
Step	A single task inside a job — run a command or use an action
Action	Reusable pre-built step from GitHub Marketplace (e.g. actions/checkout)
Runner	The VM that executes your job — GitHub-hosted or self-hosted

Hierarchy

```
Workflow
  ■■■ Job 1 (detect)
        ■■■ Step 1: Checkout
        ■■■ Step 2: Detect changed services
  ■■■ Job 2 (build-and-push)
        ■■■ Step 1: Checkout
        ■■■ Step 2: Docker login
        ■■■ Step 3: Docker build
        ■■■ Step 4: Docker push
        ■■■ Step 5: Update values.yaml
        ■■■ Step 6: Git commit and push
```

3. Workflow File Structure

```
name: Smart CI                    # Display name in GitHub UI

on:                               # Trigger conditions
  push:
    branches: [ main ]
    paths:
      - 'services/**'

jobs:                             # One or more jobs
  detect:                         # Job ID
    runs-on: ubuntu-22.04         # Runner type
    outputs:                      # Job-level outputs (pass data to other jobs)
      services: "..."
    steps:                        # List of steps
      - name: Step Name
        run: echo "hello"

  build-and-push:
    needs: detect                 # Depends on detect job (sequential)
    runs-on: ubuntu-22.04
    steps:
      - name: Another step
        run: echo "world"
```

4. Triggers (on:)

Basic push trigger

```
on:
  push:
    branches: [ main ]
```

With path filtering — critical for monorepos

```
on:
  push:
    branches: [ main ]
    paths:
      - 'services/**'       # only trigger if services/ folder changed
    paths-ignore:
      - 'k8s/**'            # ignore k8s/ (CI bot commits here — prevents loop)
```

`**` means any file at any depth. So `services/config-service/src/App.java` matches `services/**`.

> **Note:** GitHub evaluates path filters BEFORE spinning up a runner — zero cost if skipped.

Other common triggers

```
on:
  pull_request:
    branches: [ main ]

  schedule:
    - cron: '0 0 * * *'    # every day at midnight

  workflow_dispatch:         # manual trigger from GitHub UI
```

[skip ci] — prevent workflow from triggering

Any commit message containing `[skip ci]` will not trigger workflows. Used when CI bot commits back to the repo to prevent infinite loops:

```
git commit -m "ci: update image tag [skip ci]"
#                                    ↑
#              GitHub sees this and does NOT trigger the workflow
```

5. Jobs

Basic job

```
jobs:
  my-job:
    runs-on: ubuntu-22.04
    steps:
      - run: echo "hello"
```

Job dependency — sequential execution

```
jobs:
  detect:
    runs-on: ubuntu-22.04
    steps:
      - run: echo "detecting..."

  build:
    needs: detect              # waits for detect to finish
    runs-on: ubuntu-22.04      # also gives access to detect's outputs
    steps:
      - run: echo "building..."
```

By default jobs run **in parallel**. Use `needs:` to make them sequential.

Conditional job execution

```
jobs:
  build:
    needs: detect
    if: "${{ needs.detect.outputs.services != '[]' }}"   # skip if empty array
    runs-on: ubuntu-22.04
```

Job outputs — pass data to other jobs

```
jobs:
  detect:
    outputs:
      services: "${{ steps.detect_services.outputs.services }}"
    steps:
      - id: detect_services
        run: echo "services=[\"config-service\"]" >> $GITHUB_OUTPUT

  build:
    needs: detect
    steps:
      - run: echo "${{ needs.detect.outputs.services }}"
      #             ↑ reads the JSON array from detect job
```

6. Steps

Run a shell command

```
steps:
  - name: Say hello
    run: echo "hello world"
```

Multi-line command (the | pipe)

```
steps:
  - name: Multi-line
    run: |
      echo "line 1"
      echo "line 2"
      echo "line 3"
```

Use a prebuilt action

```
steps:
  - name: Checkout code
    uses: actions/checkout@v4
    with:
      fetch-depth: 0      # full git history — needed for git diff
```

Step with ID — to reference its outputs

```
steps:
  - name: Detect services
    id: detect_services
    run: echo "services=[\"config-service\"]" >> $GITHUB_OUTPUT

  - name: Use output from previous step
    run: echo "${{ steps.detect_services.outputs.services }}"
```

7. Expressions and Contexts

`${{ }}` is GitHub's expression syntax, evaluated **before** the shell runs.

Context	Example value	What it gives you
github.sha	bbaad9388fd4...	Latest commit SHA after push
github.event.before	d1d558df313c...	Previous commit SHA (before push)
github.actor	tharunbairaboina	Who triggered the workflow
github.repository_owner	tharunbairaboina	Repo owner username
github.ref	refs/heads/main	Branch/tag ref
needs.detect.outputs.services	["config-service"]	Output from detect job
steps.detect_services.outputs.x	["config-service"]	Output from a step
matrix.service	config-service	Current matrix item value
secrets.GITHUB_TOKEN	ghp_xxx...	Auto-generated scoped token

Operators in expressions

```
if: "${{ needs.detect.outputs.services != '[]' }}"   # not equal
if: "${{ github.ref == 'refs/heads/main' }}"            # equal
if: "${{ success() }}"                                   # previous job succeeded
if: "${{ failure() }}"                                   # previous job failed
```

8. Passing Data Between Steps and Jobs

$GITHUB_OUTPUT — between JOBS

```
# Write in one step
echo "services=[\"config-service\"]" >> $GITHUB_OUTPUT

# Read in another JOB
${{ needs.detect.outputs.services }}
```

$GITHUB_ENV — between STEPS in the same job

```
# Write in one step
echo "IMAGE=ghcr.io/tharunbairaboina/config-service" >> $GITHUB_ENV

# Read in next step (just like a regular shell variable)
echo $IMAGE
```

Comparison

	$GITHUB_OUTPUT	$GITHUB_ENV
Scope	Between different jobs	Between steps in the same job
Access syntax	${{ needs.jobid.outputs.key }}	$VARIABLE_NAME
Analogy	Function return value	Global variable inside a function

Note: You CAN use $GITHUB_OUTPUT for same-job step data too, but $GITHUB_ENV is simpler — no need to reference steps.xxx.outputs.xxx.

9. Matrix Strategy

Matrix spawns **one parallel job per item** in an array.

```
strategy:
  matrix:
    service: "${{ fromJson(needs.detect.outputs.services) }}"
#             ↑ fromJson() converts JSON string → actual array
#             ↑ GitHub creates one job copy per array item

services = ["config-service", "service-registry"]
     ↓
matrix expands into 2 parallel jobs:
  job 1: matrix.service = "config-service"      (fresh Ubuntu VM)
  job 2: matrix.service = "service-registry"    (fresh Ubuntu VM)
```

Dynamic vs Hardcoded

```
# WRONG — hardcoded, breaks when you add a new service
matrix:
  service: [config-service, service-registry, request-builder]

# CORRECT — dynamic, reads actual folders, works for any number
matrix:
  service: "${{ fromJson(needs.detect.outputs.services) }}"
```

With dynamic matrix, adding a new service = create folder under `services/`. Zero changes to ci.yml ever again.

10. Permissions

```
permissions:
  contents: write    # allows git push back to repo
  packages: write    # allows docker push to ghcr.io
```

Without correct permissions:

```
docker push → 403 Forbidden
git push    → 403 Forbidden
```

Default permissions are **read-only**. You must explicitly grant write access.

> **Note:** Also set globally in GitHub repo: Settings → Actions → General → Workflow permissions → Read and write

11. Secrets and Tokens

GITHUB_TOKEN — auto-generated

```
password: ${{ secrets.GITHUB_TOKEN }}
```

- GitHub **automatically creates** this at the start of every workflow run
- Scoped to current repo only
- Expires when workflow ends — you never create or manage it
- Used for: pushing to ghcr.io, git push to same repo

Custom secrets — for external services

```
# Create in: GitHub repo → Settings → Secrets → Actions → New secret
username: ${{ secrets.DOCKERHUB_USERNAME }}
password: ${{ secrets.DOCKERHUB_TOKEN }}
```

Registry comparison

Registry	Auth method	Setup needed
ghcr.io (GitHub)	GITHUB_TOKEN auto-generated	None — works out of the box
DockerHub	Custom secrets	Manual — create and store secrets
ECR (AWS)	AWS credentials	Manual — configure AWS access

12. Docker + GHCR Integration

Login to GHCR

```
- name: Login to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

# Internally this runs:
# echo "$GITHUB_TOKEN" | docker login ghcr.io -u tharunbairaboina --password-stdin
```

Build with multiple tags

```
IMAGE=ghcr.io/${{ github.repository_owner }}/${{ matrix.service }}
SHA=${{ github.sha }}

docker build \
  -t $IMAGE:$SHA \        # exact commit SHA — 100% traceable
  -t $IMAGE:latest \      # latest — always newest
  services/${{ matrix.service }}/
  # ↑ path to Dockerfile context
```

Image naming convention

```
ghcr.io / tharunbairaboina / config-service : bbaad9388fd4...
  ↑             ↑                ↑                   ↑
registry     owner/org       service name        commit SHA tag
```

Push both tags

```
docker push $IMAGE:$SHA
docker push $IMAGE:latest
```

Does ubuntu runner have Docker pre-installed?

Yes! Both `ubuntu-latest` and `ubuntu-22.04` come with Docker, git, jq, curl, kubectl, and many more tools pre-installed. No installation step needed.

13. Helm / K8s Values Update

values.yaml structure

```
image:
  repository: ghcr.io/tharunbairaboina/config-service
  tag: bbaad9388fd48388e4fa58bcef539697a6d72a87
  pullPolicy: Always
```

CI updates it with sed

```
FILE=k8s/${{ matrix.service }}/values.yaml

# Update repository line — match "repository:" + anything after it
sed -i "s|repository:.*|repository: $IMAGE|g" $FILE

# Update tag line
sed -i "s|tag:.*|tag: $SHA|g" $FILE
```

Before: `repository: 192.168.3.150:5000/config-server-repo/config-server`

After: `repository: ghcr.io/tharunbairaboina/config-service`

Commit updated values.yaml back to repo

```
git config user.name "github-actions[bot]"
git config user.email "github-actions[bot]@users.noreply.github.com"

git pull --rebase origin main        # sync before push — avoids conflicts

git add k8s/${{ matrix.service }}/values.yaml

if git diff --staged --quiet; then
  echo "No changes to commit"        # skip empty commits
else
  git commit -m "ci: update ${{ matrix.service }} image tag to $SHA [skip ci]"
  git push
fi
```

14. Real-World Issues and Fixes

Issue 1: YAML syntax error — unquoted ${{ }}

Error / Symptom

```
Invalid workflow file: .github/workflows/ci.yml#L25
You have an error in your yaml syntax on line 25
```

Cause

{ is a special YAML character. Unquoted ${{ }} confuses the YAML parser and causes it to report a syntax error on a later line.

Fix

```
# WRONG
services: ${{ steps.detect_services.outputs.services }}

# CORRECT — always quote values containing ${{ }}
services: "${{ steps.detect_services.outputs.services }}"
```

Issue 2: All services triggering instead of only changed ones

Error / Symptom

```
Changed services: config-service service-registry request-builder
# All 3 detected even when you changed only 1
```

Cause

First-commit fallback uses git ls-files which lists ALL tracked files, making every service appear changed.

Fix

```
if [ "${{ github.event.before }}" = "0000000000000000000000000000000000000000" ]; then
  CHANGED=$(git diff --name-only HEAD~1 HEAD 2>/dev/null || echo "")
else
  CHANGED=$(git diff --name-only ${{ github.event.before }} ${{ github.sha }})
fi
```

Issue 3: Zero SHA on new branch push

Error / Symptom

```
github.event.before = 0000000000000000000000000000000000000000
git diff 000...000 HEAD → fails silently → nothing detected
```

Cause

When you push a branch for the first time, 'before' is all zeros. git diff with zero SHA fails and returns nothing.

Fix

```
# Same fix as Issue 2 — check for zero SHA and use HEAD~1 as fallback
if [ "${{ github.event.before }}" = "0000000000000000000000000000000000000000" ]
```

Issue 4: git push rejected — remote rejected

Error / Symptom

```
! [remote rejected] main -> main
(cannot lock ref: is at dd6faf3 but expected 2c40cb1)
```

Cause

CI bot committed to remote while you were making local commits. Your local history diverged from remote.

Fix

```
# Quick fix
git pull --rebase origin main
git push

# Long-term fix — add to ci.yml:
on:
  push:
    paths-ignore:
      - 'k8s/**'   # CI only touches k8s/ — ignore bot commits
```

Issue 5: Why not use HEAD~1 for diff?

Error / Symptom

```
git diff --name-only HEAD~1 HEAD
# Misses commits if you push 2+ commits at once
```

Cause

If you push 3 commits at once, HEAD~1 only covers the last commit — misses the other 2. You'd need HEAD~3 but you can't know in advance.

Fix

```
# CORRECT — always covers ALL commits in the push
git diff --name-only ${{ github.event.before }} ${{ github.sha }}
# before = last commit GitHub knew about
# sha    = latest commit after your push
```

Issue 6: Runner stuck in loop / waiting

Error / Symptom

```
Waiting for a runner to pick up this job...
Job is about to start: GitHub Actions 1000000017
Waiting for a runner to pick up this job...  ← loop!
```

Cause

GitHub infrastructure issue. Runner gets assigned then dies. GitHub has had 57+ Actions outages in the past year.

Fix

```
# Fix: cancel workflow and switch runner label
runs-on: ubuntu-22.04    # more stable pool than ubuntu-latest

# Check status: https://www.githubstatus.com
```

Issue 7: fetch-depth must be 0

Error / Symptom

```
fatal: ambiguous argument 'HEAD~1': unknown revision
# git diff fails — no history available
```

Cause

Default checkout is shallow (only latest commit). git diff needs the old commit to be available locally.

Fix

```
- uses: actions/checkout@v4
  with:
    fetch-depth: 0   # full history — required for git diff
```

Monorepo CI bot conflict — solution options

Approach	How it works	Best for
paths-ignore: k8s/**	CI only touches k8s/, devs touch services/ — no overlap	Simple monorepos
[skip ci] in message	GitHub skips workflow for bot commits entirely	Any setup
Separate CI branch	Bot pushes to ci-updates branch, not main	Medium complexity
Separate GitOps repo	Code and K8s config in different repos entirely	Production / enterprise
Push with retry	git pull --rebase && git push in a loop with 3 retries	Quick temporary fix

15. The Complete ci.yml — Annotated

```
name: Smart CI
# Display name in GitHub Actions tab

on:
  push:
    branches: [ main ]
    paths:
      - 'services/**'    # trigger only if services/ changed
    paths-ignore:
      - 'k8s/**'         # ignore CI bot commits to k8s/

jobs:
  detect:
    runs-on: ubuntu-22.04
    # Fresh Ubuntu 22.04 VM — more stable than ubuntu-latest

    outputs:
      services: "${{ steps.detect_services.outputs.services }}"
      # Job-level output — passes JSON array to build-and-push job
      # Must be quoted to avoid YAML parsing issues with { characters

    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          # Full git history — needed for git diff between commits

      - name: Detect changed services
        id: detect_services
        # id allows other steps/jobs to reference this step's outputs
        run: |
          echo "Detecting changes..."

          # Handle zero SHA (brand new branch push)
          if [ "${{ github.event.before }}" = "0000000000000000000000000000000000000000" ]; then
            CHANGED=$(git diff --name-only HEAD~1 HEAD 2>/dev/null || echo "")
          else
            CHANGED=$(git diff --name-only ${{ github.event.before }} ${{ github.sha }})
          fi
          # Covers ALL commits in the push — not just the last one

          echo "Changed files:"
          echo "$CHANGED"

          SERVICES=()                  # empty bash array
          for svc_path in services/*/; do
          # Glob expands to all directories — fully dynamic
            svc=$(basename "$svc_path")
            # services/config-service/ → config-service

            if echo "$CHANGED" | grep -q "services/$svc/"; then
            # grep -q = quiet (yes/no only) — natural deduplication
              SERVICES+=("$svc")
            fi
          done

          echo "Changed services: ${SERVICES[@]}"
```

```
          JSON=$(printf '%s
' "${SERVICES[@]}" | jq -R . | jq -s -c .)
          # bash array → JSON array:
          # printf: one item per line
          # jq -R:  wrap each line in quotes
          # jq -s -c: slurp into array + compact single line
          # Result: ["config-service","service-registry"]

          echo "services=$JSON" >> $GITHUB_OUTPUT
          # Expose as step output → read by detect job outputs: block

  build-and-push:
    runs-on: ubuntu-22.04
    needs: detect
    # Wait for detect + access its outputs

    if: "${{ needs.detect.outputs.services != '[]' }}"
    # Skip entire job if nothing changed

    strategy:
      matrix:
        service: "${{ fromJson(needs.detect.outputs.services) }}"
    # fromJson() converts JSON string → actual array
    # Spawns one parallel job per changed service

    permissions:
      contents: write    # needed for git push
      packages: write    # needed for docker push to ghcr.io

    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
          # GITHUB_TOKEN auto-generated — no setup needed, expires after workflow

      - name: Build Docker image
        run: |
          IMAGE=ghcr.io/${{ github.repository_owner }}/${{ matrix.service }}
          SHA=${{ github.sha }}

          docker build \
            -t $IMAGE:$SHA \       # exact SHA tag — 100% traceable
            -t $IMAGE:latest \     # latest tag — always newest
            services/${{ matrix.service }}/

      - name: Push Docker image
        run: |
          IMAGE=ghcr.io/${{ github.repository_owner }}/${{ matrix.service }}
          SHA=${{ github.sha }}

          docker push $IMAGE:$SHA
          docker push $IMAGE:latest
```

```
    echo "IMAGE=$IMAGE" >> $GITHUB_ENV
    echo "SHA=$SHA" >> $GITHUB_ENV
    # $GITHUB_ENV makes these available in all subsequent steps
    # Access as $IMAGE and $SHA directly

- name: Update Helm values.yaml
  run: |
    FILE=k8s/${{ matrix.service }}/values.yaml
    sed -i "s|repository:.*|repository: $IMAGE|g" $FILE
    sed -i "s|tag:.*|tag: $SHA|g" $FILE
    echo "Updated values.yaml:"
    cat $FILE

- name: Commit updated values.yaml
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"

    git pull --rebase origin main   # sync before push

    git add k8s/${{ matrix.service }}/values.yaml

    if git diff --staged --quiet; then
      echo "No changes to commit"
    else
      git commit -m "ci: update ${{ matrix.service }} image tag to $SHA [skip ci]"
      git push
    fi
```

16. Bash Tools Used in CI

git diff

```
# List changed filenames between two commits
git diff --name-only HEAD~1 HEAD

# Between specific commits (what CI uses)
git diff --name-only d1d558d bbaad93

# Output:
# services/config-service/text.txt
# services/request-builder/1.tct
```

Extract parts of path

```
# Get only filenames (strip path)
git diff --name-only d1d558d bbaad93 | xargs basename
# text.txt
# 1.tct

# Get service names (2nd field when split by /)
git diff --name-only d1d558d bbaad93 | cut -d'/' -f2
# config-service
# request-builder

# Remove duplicates (if 10 files changed in same service)
git diff --name-only d1d558d bbaad93 | cut -d'/' -f2 | sort -u
# config-service  ← appears once even if 10 files changed
```

jq — JSON processor

Flag	Full name	What it does
-R	Raw input	Treat plain text as string (not JSON), wraps in quotes
-r	Raw output	Strip JSON quotes from output — gives plain string
-s	Slurp	Combine all inputs into one JSON array
-c	Compact	Single line output — no spaces or newlines

```
# Our pipeline: bash array → JSON array
printf '%s\n' "${SERVICES[@]}"    # one item per line:  config-service
| jq -R .                          # wrap in quotes:     "config-service"
| jq -s -c .                       # slurp + compact:    ["config-service","service-registry"]
```

grep

```
# -q = quiet mode (just yes/no, no output printed)
echo "$CHANGED" | grep -q "services/config-service/"
# exit 0 = found → if block runs
# exit 1 = not found → if block skipped
```

sed

```
# In-place substitution
sed -i "s|repository:.*|repository: ghcr.io/tharunbairaboina/config-service|g" values.yaml
```

```
#     ↑ edit in place
#        ↑ s = substitute
#          ↑ | delimiter (using | not / to avoid conflicts with URLs)
#                      ↑ .* = match anything after "repository:"
#                                                          ↑ g = global
```

basename

```
basename "services/config-service/"
# config-service
# Strips path — only keeps the last component
```

17. Interview Quick Reference

Key GitHub Actions concepts

Concept	One-liner
on.push.paths	GitHub native filter — no runner spun up if paths don't match
outputs:	Pass data from one job to another job
needs:	Make jobs run sequentially + share outputs
if:	Conditionally run a job or step
strategy.matrix	Spawn N parallel jobs from an array dynamically
$GITHUB_OUTPUT	Expose step data to other jobs
$GITHUB_ENV	Share variables between steps in same job
GITHUB_TOKEN	Auto-generated scoped token — no setup needed, expires after workflow
fetch-depth: 0	Full git history — required for git diff to work
[skip ci]	Prevent workflow trigger on a specific commit
permissions:	Scope what GITHUB_TOKEN can do (contents:write, packages:write)
paths-ignore:	Ignore certain file changes from triggering the workflow

$GITHUB_OUTPUT vs $GITHUB_ENV

```
$GITHUB_OUTPUT → between JOBS  → needs.jobid.outputs.key
$GITHUB_ENV    → between STEPS  → $VARIABLE_NAME  (simpler access)
```

How github.event.before and github.sha work

```
Before push:   A → B → C           (github.event.before = C)
After push:    A → B → C → D → E   (github.sha = E)

git diff C..E = all files changed across commits D and E
               covers ALL commits in one push — not just the last one
```

CI platforms and their before SHA variables

Platform	Before SHA	Current SHA
GitHub Actions	github.event.before	github.sha
GitLab CI	CI_COMMIT_BEFORE_SHA	CI_COMMIT_SHA
Jenkins	GIT_PREVIOUS_COMMIT	GIT_COMMIT
Bitbucket	No native variable	BITBUCKET_COMMIT

Platform	Before SHA	Current SHA
Azure DevOps	Manual merge-base calculation	Build.SourceVersion

Complete CI flow — end to end

```
Push to main (services/** changed)
          ↓
GitHub checks paths filter (free — no runner cost)
          ↓
detect job spins up (Ubuntu VM)
          ↓
git diff github.event.before → github.sha
→ finds exactly which service folders changed
          ↓
loop through services/*/ dynamically
→ builds JSON array ["service-registry"]
→ writes to $GITHUB_OUTPUT
          ↓
detect job ends
          ↓
services != [] ? → YES
          ↓
matrix spawns 1 parallel job per changed service
          ↓
Each job (fresh Ubuntu VM):
  → docker login ghcr.io (GITHUB_TOKEN)
  → docker build services/service-registry/Dockerfile
  → docker push ghcr.io/tharunbairaboina/service-registry:SHA
  → docker push ghcr.io/tharunbairaboina/service-registry:latest
  → sed updates k8s/service-registry/values.yaml tag
  → git commit [skip ci] + git push
          ↓
ArgoCD / FluxCD detects values.yaml change
          ↓
K8s pulls new image → rolling update → redeploy ■
```

Document based on real implementation of a monorepo Smart CI pipeline for 3 Java microservices using GitHub Actions, GHCR, Helm, and Kubernetes.